

```

1  (*
2      CS 51 Lab 19
3      Brian's Brain Cellular Automaton
4      https://en.wikipedia.org/wiki/Brian's_Brain
5  *)
6
7  module G = Graphics ;;
8
9  (* Automaton parameters *)
10 let cGRID_SIZE = 100 ;; (* width and height of grid in cells *)
11 let cSPARSITY = 5 ;; (* inverse of proportion of cells initially live *)
12
13 (* Rendering parameters *)
14 let cCOLOR_LIVE = G.rgb 200 200 200 ;; (* color to depict live cells *)
15 let cCOLOR_DYING = G.rgb 130 0 0 ;; (* color to depict dying cells *)
16 let cCOLOR_DEAD = G.rgb 0 0 0 ;; (* background color *)
17 let cCOLOR_LEGEND = G.rgb 173 106 108 ;; (* color for textual legend *)
18 let cSIDE = 8 ;; (* width and height of cells in pixels *)
19 let cRENDER_FREQUENCY = 1 (* how frequently grid is rendered (in ticks) *) ;;
20 let cFONT = Some "-adobe-times-bold-r-normal--34-240-100-100-p-177-iso8859-9"
21
22 (*-----
23 Brian's Brain Cellular Automaton
24 *)
25
26 (*.....
27 States, updates, and utility functions
28 *)
29
30 (* We take the opportunity to abstract the cell states as an
31 enumerated type *)
32 type bbrain_state =
33 | Live
34 | Dying
35 | Dead ;;
36
37 (* offset index off -- Returns the `index` offset by `off` allowing
38 for wraparound. *)
39 let rec offset (index : int) (off : int) : int =
40   if index + off < 0 then
41     cGRID_SIZE - (offset ~-index ~-off)
42   else
43     (index + off) mod cGRID_SIZE ;;
44
45 (* bbrain_update grid i j -- Returns the updated value for cell at `i,
46 j` in the grid based on rules of Brian's Brain
47 <https://en.wikipedia.org/wiki/Brian%27s_Brain>. *)
48 let bbrain_update (grid : bbrain_state array array)
49   (i : int) (j : int)
50   : bbrain_state =
51   let neighbors = ref 0 in
52   for i' = ~-1 to ~+1 do

```

```

53   for j' = ~-1 to ~+1 do
54     let i_ind = offset i i' in
55     let j_ind = offset j j' in
56     neighbors := !neighbors
57               + match grid.(i_ind).(j_ind) with
58                 | Live -> 1
59                 | Dying -> 0
60                 | Dead -> 0
61   done
62 done;
63 (* determine new state based on neighbor count *)
64 let new_state =
65   match grid.(i).(j) with
66   | Live -> Dying
67   | Dying -> Dead
68   | Dead -> if !neighbors = 2 then Live else Dead in
69   new_state ;;
70
71 (* bbrain_random_state () -- Returns a random cell state, either
72    Living or Dead, but not Dying (as those will get generated on the
73    next time step). *)
74 let bbrain_random_state () =
75   let states = [Live; Dead] in
76   List.nth states (Random.int (List.length states))
77
78 (* .....
79    Making Brian's Brain
80    *)
81
82 (* Automaton specification *)
83
84 module BBrainSpec : (Cellular.AUT_SPEC
85                     with type state = bbrain_state) =
86   struct
87     type state = bbrain_state
88     let initial : state = Dead
89     let grid_size : int = cGRID_SIZE
90     let random_state = bbrain_random_state
91     let update = bbrain_update
92     let name : string = "Brain's Brain"
93     let cell_color (cell : state) : G.color =
94       match cell with
95       | Live -> cCOLOR_LIVE
96       | Dying -> cCOLOR_DYING
97       | Dead -> cCOLOR_DEAD
98     let side_size : int = cSIDE
99     let legend_color : G.color = cCOLOR_LEGEND
100    let font : string option = cFONT
101    let render_frequency : int = cRENDER_FREQUENCY
102  end ;;
103
104 module Aut = Cellular.Automaton (BBrainSpec) ;;
105
106 (* .....

```

```

107   Running the automaton and displaying the results
108   *)
109
110   (* main seed -- Runs the automaton using the provided random `seed`
111      (for replicability), displaying updates to the graphics window. *)
112   let main seed =
113
114       Random.init seed;      (* initialize randomness *)
115       Aut.graphics_init ();  (* ... and graphics window *)
116
117       let initial_grid = Aut.random_grid () in
118       Aut.run_grid initial_grid ;;
119
120   (* Run the automaton with user-supplied seed *)
121   let _ =
122       if Array.length Sys.argv <= 1 then
123           Printf.printf "Usage: %s <seed>\n where <seed> is integer seed\n"
124                         Sys.argv.(0)
125       else
126           main (int_of_string Sys.argv.(1)) ;;

```